WALCHAND COLLEGE OF ENGINEERING, SANGLI

Department of Computer Science & Engineering

LAB ASSIGNMENT 1 (LA-1)

Machine Learning Model for Robotics Perception

Real-Time Human Detection and Presence Monitoring System

Name - Atharv Kulkarni

PRN - 245109074

Class - TY CSE(B)

Academic Year: 2026–2027

Project Title

Real-Time Human Detection and Presence Monitoring using MobileNetV2, Haar Cascades, and Intel RealSense

GitHub Repository: https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/LA_1_ML_Model

AIM

To develop and evaluate a vision-based human-presence detection system combining a fine-tuned MobileNetV2 classifier, multi-pose Haar-cascade detection, and temporal filtering, with support for standard RGB cameras and Intel RealSense RGB/depth streams.

OBJECTIVES

- Set up the Python environment with PyTorch, Torchvision, OpenCV, and the optional Intel RealSense SDK.
- Prepare and evaluate an RGB dataset containing human and non-human examples.
- Fine-tune a pretrained MobileNetV2 model for binary human/non-human classification.
- Combine frontal-face, profile-face, and upper-body cues with neural-network predictions.
- Reduce false absence alerts during short occlusions using a temporal grace filter.
- Display live presence status, tracking cues, session time, FPS, and warning indicators in an OpenCV HUD.

THEORY

1. Human Detection and Robotics Perception

Robotics perception converts camera observations into information used by a system to make decisions. Human-presence detection classifies each image or video frame as containing a person or a non-human scene. Reliable operation must account for pose changes, partial occlusion, lighting variation, and movement.

2. MobileNetV2 and Transfer Learning

MobileNetV2 is a lightweight convolutional neural network designed for efficient inference. Transfer learning reuses visual features learned from a large dataset and adapts final layers to target classes. Here, early feature layers are frozen, the final feature block is fine-tuned, and a custom two-class classifier predicts HUMAN or NON_HUMAN.

3. Haar Cascades and Decision Fusion

OpenCV Haar cascades detect frontal faces, profile faces, and upper bodies. The implementation checks multiple pose cues, including a horizontally flipped frame for the opposite profile. These physical cues are combined with MobileNetV2 probabilities as a fallback when face or torso detection is ambiguous.

4. Temporal Absence Grace Filter

A single missed frame should not immediately mark a candidate absent. A 20-frame grace buffer (approximately 0.7–1.0 seconds depending on frame rate) retains the prior presence state during brief missed detections. A positive detection resets the absence counter.

5. Intel RealSense RGB and Depth

The Intel RealSense pipeline can provide synchronized color and depth frames. The included notebook demonstrates device discovery, stream capture, and depth colorization. The reported live classification uses the RGB frame; it should not be described as depth-based classification unless depth data is explicitly incorporated into the model input.

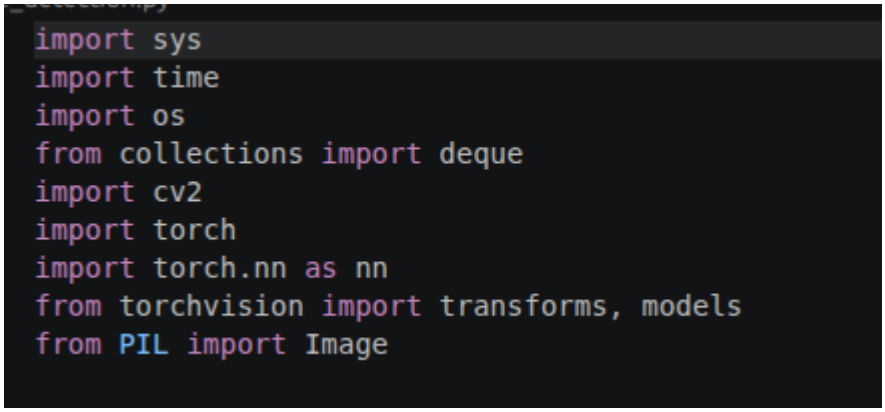
PROJECT FILE BREAKDOWN

File / Folder	Role and Functionality
train.py	Fine-tunes MobileNetV2 using augmentation, AdamW, label-smoothed cross-entropy, and cosine learning-rate scheduling.
test_images.py	Runs batch inference on the RGB dataset and reports per-image confidence and overall accuracy.
live_detection.py	Real-time camera monitoring using multi-pose Haar cascades, neural-network fallback, temporal filtering, and a HUD.
realsense_dataset/	RGB and depth samples grouped into human and non_human classes.
Intel_Realsense_Cammera_Code.ipynb	Demonstrates RealSense device access, synchronized color/depth acquisition, and depth visualization.
human_detector.pth	Saved PyTorch model weights used for inference.

EXPERIMENT STEPS WITH SCREENSHOT OF EACH STEP

Step 1: Installation and Environment Setup

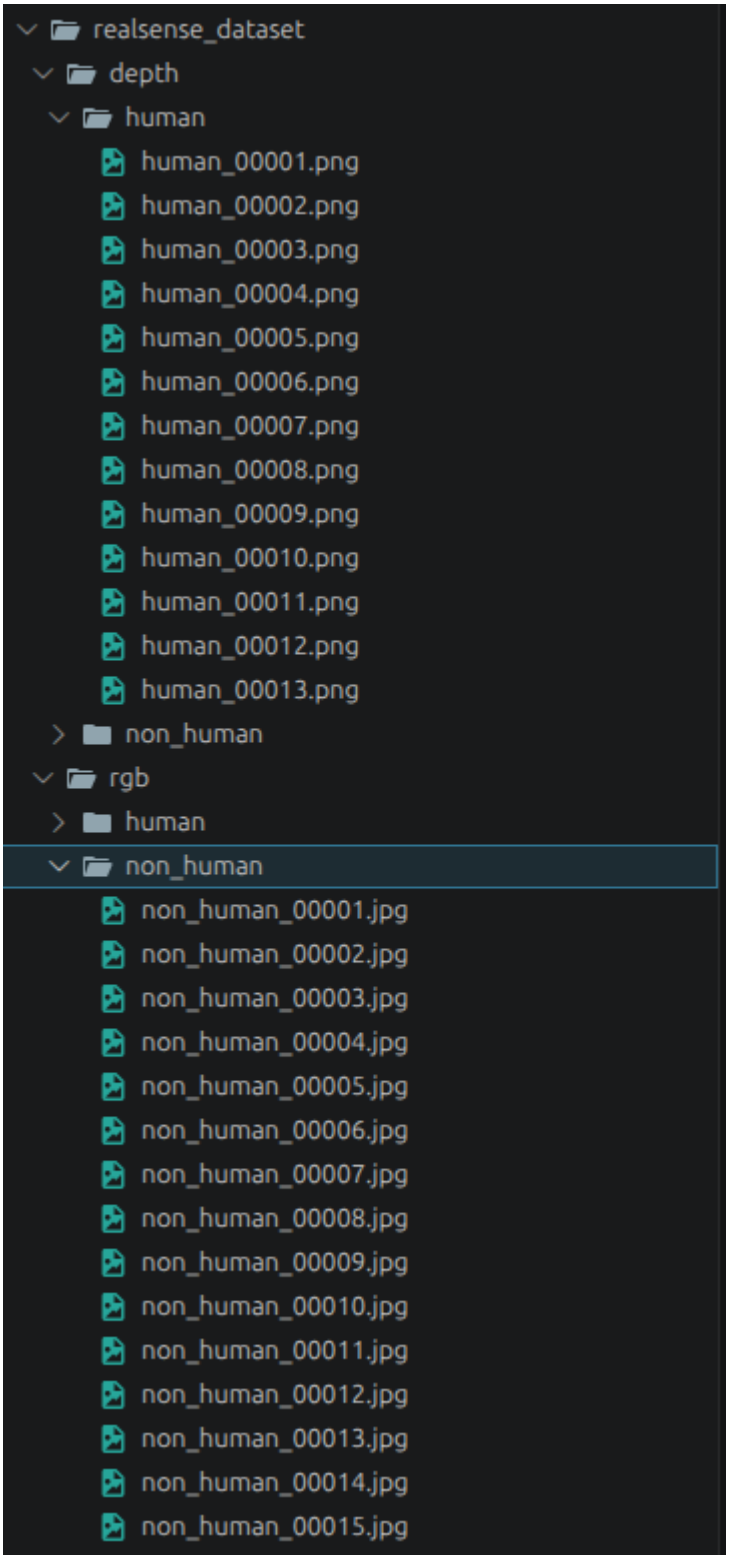
Set up the Anaconda/Python environment and install PyTorch, Torchvision, OpenCV, Pillow, NumPy, and Matplotlib. Install pyrealsense2 when testing the RealSense notebook. Confirm the dataset and scripts are available.



```
import sys
import time
import os
from collections import deque
import cv2
import torch
import torch.nn as nn
from torchvision import transforms, models
from PIL import Image
```

Step 2: RealSense Camera Integration

Run Intel_Realsense_Cammera_Code.ipynb to discover the device, configure RGB/depth streams, capture synchronized frames, and visualize colorized depth. Compatible RealSense hardware is required.


realsense_dataset
depth
human
human_00001.png
human_00002.png
human_00003.png
human_00004.png
human_00005.png
human_00006.png
human_00007.png
human_00008.png
human_00009.png
human_00010.png
human_00011.png
human_00012.png
human_00013.png
non_human
rgb
human
non_human
non_human_00001.jpg
non_human_00002.jpg
non_human_00003.jpg
non_human_00004.jpg
non_human_00005.jpg
non_human_00006.jpg
non_human_00007.jpg
non_human_00008.jpg
non_human_00009.jpg
non_human_00010.jpg
non_human_00011.jpg
non_human_00012.jpg
non_human_00013.jpg
non_human_00014.jpg
non_human_00015.jpg

Step 3: Model Training (train.py)

Run train.py to load the RGB dataset, apply augmentation, fine-tune MobileNetV2 and its classifier, and save the weights as human_detector.pth.

train.py > ...

You, last month | author (You)

```python
import os      You, last month • done robotics assignments …
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models

# 1. Advanced Data Augmentation for Small Datasets
train_transform = transforms.Compose([
    transforms.Resize((240, 240)),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.05),
    transforms.RandomAffine(degrees=0, translate=(0.08, 0.08)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# 2. Load Dataset
data_dir = 'realsense_dataset/rgb'
dataset = datasets.ImageFolder(data_dir, transform=train_transform)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=4, shuffle=True, drop_last=False)

print(f"Loaded {len(dataset)} images belonging to classes: {dataset.classes}")

# 3. Setup Pretrained MobileNetV2 with Fine-Tuning
weights = models.MobileNet_V2_Weights.DEFAULT
model = models.mobilenet_v2(weights=weights)

# Freeze base features, but unfreeze the last high-level feature block for fine-tuning
for param in model.parameters():
    param.requires_grad = False

# Unfreeze the last feature layer (block 18) for better human texture/shape adaptation
for param in model.features[17:].parameters():
    param.requires_grad = True

# Replace the classifier head
model.classifier[1] = nn.Sequential(
    nn.Dropout(p=0.2),
    nn.Linear(model.last_channel, 2)
)
```

Step 4: Dataset Evaluation (test_images.py)

Run test_images.py against realsense_dataset/rgb/ and inspect predictions and confidence for human/non-human samples. Record correct predictions and overall accuracy.

test_images.py > ...

You, last month | author (You)

```python
import os      You, last month • done robotics assignments …
import glob
import torch
import torch.nn as nn
from torchvision import transforms, models
from PIL import Image

def test_all():
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Testing with device: {device}")

    # Load model
    model = models.mobilenet_v2(weights=None)
    model.classifier[1] = nn.Sequential(
        nn.Dropout(p=0.2),
        nn.Linear(model.last_channel, 2)
    )
    model.load_state_dict(torch.load('human_detector.pth', map_location=device, weights_only=True))
    model.to(device)
    model.eval()

    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    classes = ['human', 'non_human']

    # Test Human samples
    human_imgs = sorted(glob.glob('realsense_dataset/rgb/human/*.jpg'))
    non_human_imgs = sorted(glob.glob('realsense_dataset/rgb/non_human/*.jpg'))

    print("\n" + "="*50)
    print("           EVALUATING HUMAN DATASET")
    print("="*50)
    correct_human = 0
    for path in human_imgs:
        img = Image.open(path).convert('RGB')
        tensor = transform(img).unsqueeze(0).to(device)
        with torch.no_grad():
            outputs = model(tensor)
            prob = torch.softmax(outputs, dim=1)[0]
            conf, pred = torch.max(prob, dim=0)
            predicted_class = classes[pred.item()]
            is_correct = (predicted_class == 'human')
            if is_correct: correct_human += 1
            status = "PASS" if is_correct else "FAIL"
```

PROJECT EXECUTION & TEST RESULTS

The project dependencies were installed in the Anaconda environment and the training, evaluation, and live-inference scripts were executed. The following results reproduce the supplied run outputs.

1. Dataset Evaluation (test_images.py)

The model was evaluated on 28 RGB images: 13 human and 15 non-human samples. All samples were classified according to their listed ground-truth class in this run.

Dataset Class	Samples	Correct	Accuracy
Human	13	13	100.0%
Non-human	15	15	100.0%
Overall	28	28	100.0%

Human confidence scores: 98.2%, 97.2%, 95.8%, 94.5%, 99.2%, 95.8%, 98.5%, 53.4%, 98.8%, 99.2%, 99.3%, 93.7%, 99.2%.

Non-human confidence scores: 87.3%, 91.9%, 70.2%, 74.1%, 94.5%, 91.6%, 88.1%, 91.9%, 90.9%, 94.7%, 97.2%, 90.5%, 95.5%, 85.8%, 96.8%.

Interpretation: The reported benchmark achieved 28/28 correct predictions on this small dataset. human_00008.jpg was classified correctly at 53.4% confidence, so its lower confidence merits review. This result applies only to the supplied test set and does not establish 100% accuracy on unseen real-world data.

2. Model Training Pipeline (train.py)

- Architecture: MobileNetV2 with final feature block (Block 18 / features[17:]) fine-tuned and a custom two-class classifier head.
- Augmentation: random resized crop, rotation, color jitter, affine translation, and horizontal flipping.
- Optimization: AdamW, cosine annealing over 30 epochs, and label smoothing.
- Output: weights saved to human_detector.pth.

Epoch	Training Loss	Training Accuracy
01/30	0.6971	50.0%
10/30	0.3397	89.3%
20/30	0.2201	96.4%
30/30	0.3157	89.3%

The final training loss and accuracy are worse than the reported epoch-20 values. Training variability or overfitting/instability may be involved; check validation metrics before drawing conclusions about generalization.

3. Live Hardware and Inference Verification

- Camera device: /dev/video0.
- Captured frame dimensions: 640 × 480.
- Live inference output: Human = 83.2%; Non-Human = 16.8%.
- The frame was reported as a candidate detected by the system.

Command used to launch the live monitor:

```
/home/atharv/anaconda3/bin/python live_detection.py
```

Press q in the video display window to stop the session.

KEY TECHNICAL HIGHLIGHTS

Lightweight model: MobileNetV2 uses inverted residual blocks and depthwise separable convolutions for efficient inference.

Multi-pose detection: Frontal/tilted face, left/right profile, and upper-body cascades cover common candidate poses.

Neural-network fallback: The classifier provides a secondary presence signal when classical cues are ambiguous.

Absence grace filter: A 20-frame buffer helps prevent brief missed detections from immediately switching the state to absent.

Proctoring HUD: Displays candidate presence, session timer, FPS, bounding-box tags, and an alert when the candidate leaves the frame.

CONCLUSION

The project implements a real-time human-presence monitoring pipeline combining a fine-tuned MobileNetV2 classifier, multi-pose Haar-cascade detection, and temporal filtering. In the reported evaluation, all 28 provided RGB samples were classified correctly, and a live camera test produced a human probability of 83.2%. The RealSense notebook supports synchronized RGB/depth stream inspection. Because the dataset is small and final training metrics fluctuate, the 100% test accuracy is a result on this particular dataset rather than a guarantee of real-world performance. Further evaluation on a larger independent dataset and validation of depth usage in the inference pipeline are recommended.

GitHub Repository

Project source code, setup instructions, architecture, and benchmark documentation:

https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/LA_1_ML_Model/